

Netty学以致用 之



基于WebSocket的网络聊天室

北京理工大学计算机学院
金旭亮



服务端开发

关于WebSocket



WebSocket是一种常用的Web双向通讯协议，在单个TCP的连接上进行全双工通信。



WebSocket协议内建于支持HTML5的浏览器，基于WebSocket API的浏览器（客户端）与服务器之间只需要完成一次握手操作，两者之间就可以直接创建持久性的连接，进行全双工的双向数据传输。



WebSocket协议在基于HTTP协议与服务器端建立连接后，就会抛开HTTP协议直接通过TCP协议来完成实际操作了。

Netty与WebSocket



Netty官方提供了相应的组件，实现了WebSocket协议。



使用Netty的WebSocket组件，只需要向Channel所关联的Pipeline中加入相应的内置Handler，就能加载WebSocket基础功能的标准实现，开发者只需要编写一个自己的Handler，定义与客户端交互哪些信息即可。



WebSocket客户端，通常都是网页，直接使用标准的JavaScript相关对象即可，这些都是通用的。



与其它开发框架（比如Socket.io和SignalR）相比，Netty的WebSocket组件仅实现了基本的功能，不如那些框架功能多，但兼容性更好，可控性更强，开发者可依据自己需求选择合适的组件。

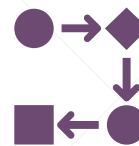
服务端入口代码



```
public class WSServer {  
    public static void main(String[] args) throws InterruptedException {  
        EventLoopGroup mainGroup = new NioEventLoopGroup();  
        EventLoopGroup subGroup = new NioEventLoopGroup();  
        try {  
            ServerBootstrap serverBootstrap = new ServerBootstrap();  
            serverBootstrap.group(mainGroup, subGroup)  
                .channel(NioServerSocketChannel.class)  
                //在WSServerInitializer中构建处理管线 (pipeline)  
                .childHandler(new WSServerInitializer());  
  
            //在8088端口监听  
            ChannelFuture future = serverBootstrap.bind(inetPort: 8088).sync();  
  
            future.channel().closeFuture().sync();  
        } finally {  
            mainGroup.shutdownGracefully();  
            subGroup.shutdownGracefully();  
        }  
    }  
}
```

这些都是通用的Netty典型代码，没有什么特别的。

构建WebSocket请求处理管线



```
1 usage
public class WSServerInitializer extends ChannelInitializer<SocketChannel> {
    2 usages
    @Override
    protected void initChannel(SocketChannel ch) throws Exception {
        var pipeline=ch.pipeline();
        //-----HTTP相关支持-----
        pipeline.addLast(new HttpServerCodec());
        //支持分块写数据流
        pipeline.addLast(new ChunkedWriteHandler());
        //聚合了FullHttpRequest或FullHttpResponse,经常用到。
        pipeline.addLast(new HttpObjectAggregator( maxContentLength: 1024*64));

        //-----WebSocket支持-----
        pipeline.addLast(new WebSocketServerProtocolHandler( websocketPath: "/ws"));
        //自定义Handler
        pipeline.addLast(new ChatHandler());
    }
}
```

这个组件，实现了
WebSocket标准协议

关于示例中用到的Netty内置Handler-1



HttpServerCodec: 实现HTTP消息的解码与编码。在它之后的Handler, 可以通过HttpObject来接收解码后的HTTP消息。



ChunkedWriteHandler: 对大数据传输的支持, 这个Handler定义了一种“ChunkedFile”, 它可以分块读取一个很大的文件, 然后再发给下一个Handler。



HttpObjectAggregator: 将HttpMessage和HttpContent组合起来, 得到一个FullHttpRequest或FullHttpResponse对象, 供下一个Handler处理。它最关键的特性, 是能处理“chunked” (分块传输) 的数据。

关于示例中用到的Netty内置Handler-2



WebSocketServerProtocolHandler: 处理WebSocket通讯细节, 将数据转换为WebSocketFrame, 传给下一个Handler。



WebSocketFrame是一个抽象类, 它的子类包括:

- © BinaryWebSocketFrame (io.netty.handler.codec.http.websocketx)
- © CloseWebSocketFrame (io.netty.handler.codec.http.websocketx)
- © ContinuationWebSocketFrame (io.netty.handler.codec.http.websocketx)
- © PingWebSocketFrame (io.netty.handler.codec.http.websocketx)
- © PongWebSocketFrame (io.netty.handler.codec.http.websocketx)
- © TextWebSocketFrame (io.netty.handler.codec.http.websocketx)



在实际开发中, 其实主要处理的就是上述子类, 了解这些子类的职责, 需要了解WebSocket协议。基本方式就是针对不同的子类, 编写相应的Handler, 进行后继处理。

实现消息的转发

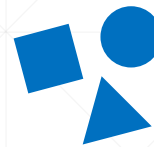


```
public class ChatHandler extends SimpleChannelInboundHandler<TextWebSocketFrame> {  
    //一个用于保存在线用户Channel的集合  
    3 usages  
    private static ChannelGroup clients=new DefaultChannelGroup(GlobalEventExecutor.INSTANCE);  
    1 usage  
    @Override  
    protected void channelRead0(ChannelHandlerContext ctx, TextWebSocketFrame msg) throws Exception {  
        //获取客户端发来的消息  
        String content=msg.text();  
        System.out.println("客户端发来: "+content);  
        //群发消息  
        for ( var channel: clients) {  
            channel.writeAndFlush(new TextWebSocketFrame(LocalTime.now()+":"+ content));  
        }  
    }  
    @Override  
    public void handlerAdded(ChannelHandlerContext ctx) throws Exception {  
        clients.add(ctx.channel());  
    }  
    @Override  
    public void handlerRemoved(ChannelHandlerContext ctx) throws Exception {  
        clients.remove(ctx.channel());  
        System.out.println("客户端"+ctx.channel().id().asLongText()+"断开。");  
    }  
}
```

WebSocket消息有特定的格式，Netty提供了TextWebSocketFrame这个类型包装数据。

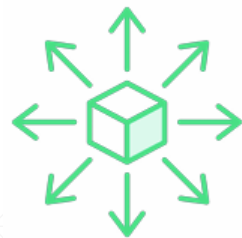
网络聊天的基本功能就是消息的广播，为了实现它，示例使用了一个集合，将所有在线用户的Channel对象引用保存起来。

ChannelGroup



- ✓ 对于服务端应用，经常需要保存所有在线的客户端通道，为此，Netty提供了一个ChannelGroup接口，以便简化代码。
- ✓ ChannelGroup的默认实现是DefaultChannelGroup，本质上是一个Channel的Set集合（不允许出现重复元素），当一个通道被关闭时，它会自动从此集合中移除，而不需要开发者手写代码。
- ✓ 可以将ServerChannel和客户端Channel一起加入到同一个ChannelGroup中，当有消息来时，ServerChannel是第一个有机会处理的，之后才是其它的客户端通道。
- ✓ 调用ChannelGroup对象的write方法，可以向集合中的所有Channel广播消息，轻松实现消息的群发。（注意，一个Channel可以加入多个Group）
- ✓ 调用ChannelGroup对象的close方法，可以一次性关闭所有通道。

一些准备工作



为了让同一个局域网内的客户端都能连上服务器，需要事先确定好服务器的IP地址：

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . :  
IPv6 地址 . . . . . : 2408:8207:1938:9010:b1ef:ad1e:a845:fa96  
临时 IPv6 地址 . . . . . : 2408:8207:1938:9010:715d:ee9e:b594:e58d  
临时 IPv6 地址 . . . . . : 2408:8207:1938:9010:952a:45f8:ebcd:d8a9  
临时 IPv6 地址 . . . . . : 2408:8207:1938:9010:fd90:9ace:b53e:13d1  
本地链接 IPv6 地址 . . . . . : fe80::e674:f53f:76dc:2134%3  
IPv4 地址 . . . . . : 192.168.1.11  
子网掩码 . . . . . : 255.255.255.0  
默认网关 . . . . . : fe80::1%3  
192.168.1.1
```

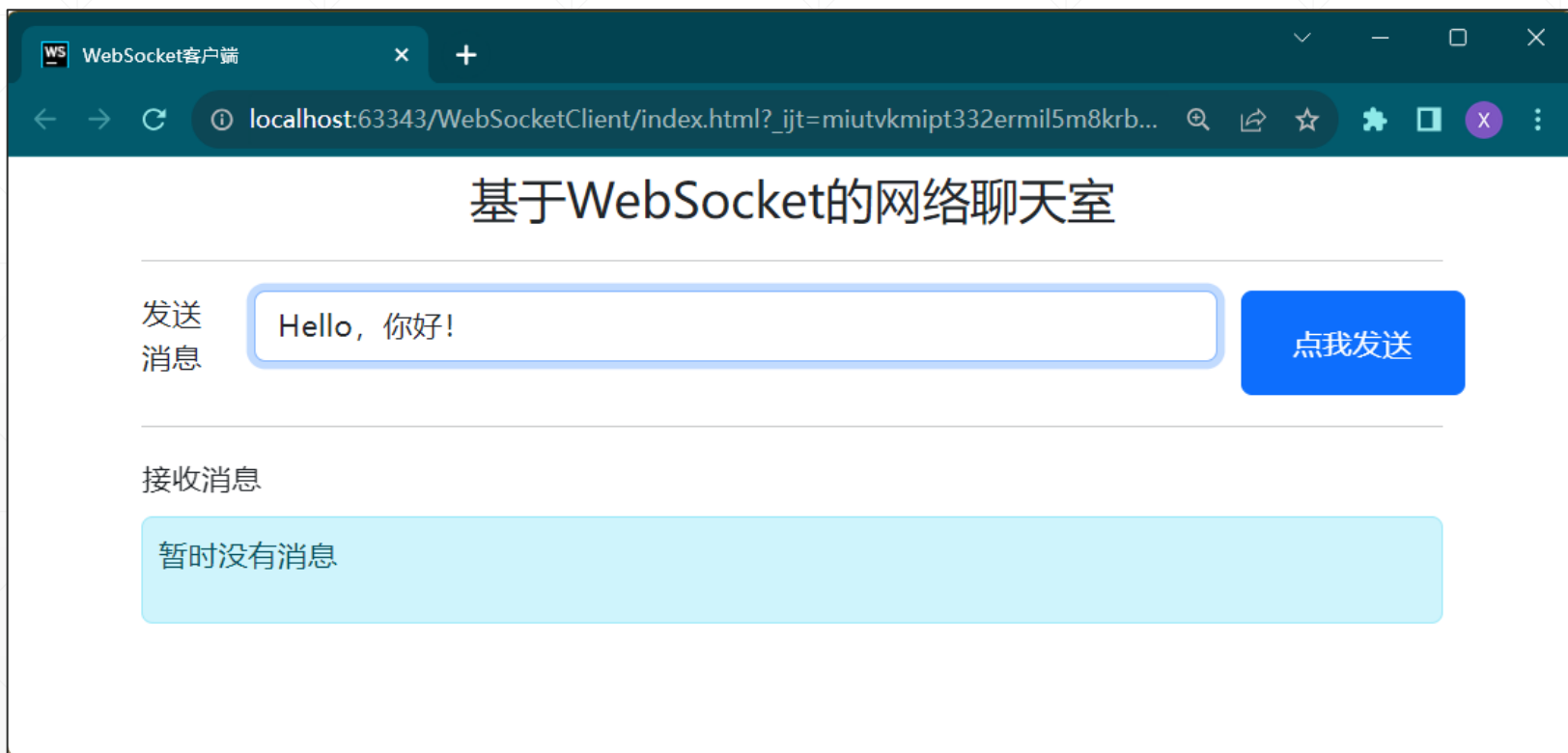


客户端开发

概述



示例使用Vue3选项式API + Bootstrap开发，非常简单，就是一个输入框：



代码

```
<script>
  let socket;
  let app = Vue.createApp({
    data() {
      return {
        msg: '',
        messages: []
      }
    },
    mounted() {...},
    methods: {
      chat() {
        socket.send(this.msg);
        this.msg = '';
      }
    }
  }).mount("#app");
</script>
```



```
mounted() {
  socket = new WebSocket("ws://192.168.1.11:8088/ws");
  socket.onopen = function () {
    console.log("onopen")
  };
  socket.onclose = function () {
    console.log("onclose")
  };
  socket.onerror = function () {
    console.log("onerror")
  };
  socket.onmessage = function (e) {
    app.messages.push(e.data);
  };
},
```

测试



将其网页部署到Web服务器上，PC或手机访问，即可进行网络聊天.....

